読み上げ台本(日本語) — WireGuard for Apache NuttX (Community Over Code, Glasgow)

本番は英語で話す前提(デッキは英語、coc-glasgow-outline.md より "English, ~20-30 min")。この日本語版は英語版と 1対1で対応する**リハーサル・理解確認用**の台本 — 内容を体に入れる、または国内で 日本語版として話す場合のベース。英語版が更新されたら、この対応も崩れるので 両方直すこと。

43枚・デモ90秒を含めて合計約33分(33:08)。2026-10-10、目次を7部(概要と背景 / 課題とアイデア / 計画 / 第1版 / 第2版 / ライブデモ / まとめ)に組み直し、第1版の動画と謝辞の2枚を追加。2026-10-10、ノートに小ネタ4つ(14・20・33・36、斜体で出典を付記)。知らない人には普通の文として聞こえるものだけで、外しても流れは変わらない。時間は累積で、英語版と同じ値(英語の読み上げ速度で計算)。2026-10-10、デモ構成図とデモの2枚に差し替え(スタックちゃんのライブデモ)。2026-10-09、経緯(タイムライン)と他手段との比較の2枚を追加し、画面の英語を易しくした。2026-10-01、前半(Background章)の 大幅拡張と、章の変わり目ごとの「目次に戻る」スライド7枚の追加を反映して全面刷新。2026-10-11、レビューを受けて: 落とし穴1(`SO_RCVTIMEO`)を2に統合して全6件に、31〜33の読み上げを短縮(細部は Q&A へ)、第1版の動画は静止画に、自己紹介のレイアウトを変更。

重心はスライド31〜33(検証) — 約5.7分をこの3枚が占める。査読者が実際に確認してくる主張はここに集中している。**読み上げはすでに短くしてある** (2026-10-11。メモリマップ、usrsock の手順、hostfs の細部などは Q&A へ移した)ので、**主張と注記(FLAT / QEMU / sim、rebase 前、PROTECTED 8回中2回の失敗は wg のスタック不足で修正済み(QEMU のみ)、4 CPU は1回の スクリプト実行、75秒見ていて)はこれ以上削らない。**削るならこの3枚の前か後。31→32→33は意味的にも一体: 31「どこで動くか」→32「動いたことは何を証明しないか」→ 33「唯一、判断で答えた場所」。分割すると論旨が崩れる。

スライド2と4〜15(About Me、Overview〜Architecture。経緯と比較の2枚を含む)も削らない。NuttXとは何か、どこで 使われているか、Sonyでの歴史、自分がどこで出会ったか、そこからWireGuardに至った 理由 — この因果の連鎖が、後半すべての前提になる。

時間が足りないときに削る順番:

1. スライド40(Giving back: where it stands — 還元の現状報告は本編の主張を支える**おまけ**で、背骨ではない。真っ先に削ってよい。削る場合は Q&A で聞かれたら答える程度に留める)
2. スライド39(移植性の表 — 補強であって背骨ではない)
3. スライド38(運用性)

ライブデモが止まったら、すぐローカル録画へ切り替える。**目次に戻るスライド(3・11・16・19・27・34・37)は各10秒程度の一呼吸** — 読み上げる ものではなく、指し示して一拍置くだけ。本当に時間がなければ無言で次へ送ってもよい (時間短縮にはほぼ寄与しないので、切るのはここではなく上の3枚から)。

本番前に確認: (1) 合計約33分は30分枠を超えている。本編は27〜28分を目標にリハーサルし、上の削る順番を本番前に決めておく。(2) GitHubスター数(FreeRTOS/Zephyr/NuttXの比較スライド)は変動するので、本番直前に実数を再確認するか、スライド上の日付表記をそのまま残す。

---

1 — タイトル (0:00 → 0:47)

画面: "WireGuard for Apache NuttX"。ESP32-S3 と SPRESENSE が実 Wi-Fi でトンネル / 実際の WireGuard ピア (Linux kernel, Windows クライアント) / QEMU 上のカーネルビルドで動作 (BUILD_KERNEL) / NuttX 13.0.1・master・12.7.0 が同一ソース。

話す:

- 今日は WireGuard を Apache NuttX に持ち込む話をします。
- これは `wg0` というネットワークデバイスとして実装した WireGuard VPN です。ESP32-S3 と SPRESENSE、両方とも実 Wi-Fi 経由でトンネルが立っています。ピアは 常に本物の WireGuard 実装 — Linux カーネルモジュールと公式 Windows クライアント — です。apps版は同じソースで NuttX の3バージョンに対応します。話の後半では、新しいドライバが QEMU 上の KERNEL / PROTECTED ビルドで、そして両ボードの通常の FLAT ビルドで動きます。ここが今日の、より難しく、より 面白い半分です。
- ただ、この話の主眼は「動いた」ことではありません。「動く」が何を隠していたか、そしてカーネルに押し込んだことで何が表に出たか、という話です。

2 — 自己紹介 (0:47 → 1:37)

画面: 見出し「From Japan: I work at Sony, and I build things as a hobby.」。上: 本人写真・ 名前の1行・SNS の狐アイコン。下の左: 日本地図(① 名古屋 生まれ → ② 仙台 大学 → ③ 川崎 現在)。 下の右: Hobby engineer の写真5枚(Space: モデルロケット / Semiconductors: ウェハ上の試作チップ / DNA nanotech: cryo-TEM 像 / Robots: ハッカソンのヒューマノイド / Linux: LinuC のペンギン)と、 活動のチップ3つ。経歴の流れ(分子ロボティクス → … → NuttX)は画面にはなく、口頭だけ。

話す:

- 簡単に自己紹介します — どこから来たか、という話です。
- Sony Semiconductor Solutions で edge AI エンジニアをしていて、application 側から Apache NuttX に関わっています。日本から来ました: 名古屋生まれ、大学は仙台、 今は川崎に住んでいます。仕事の外ではホビーエンジニアで、宇宙・半導体・ ロボットまわりのものを作っています。大学では DNA でナノ構造を作り、Linux も大好きです。SNS では、この写真の狐が私です。ここに至る道のりは、分子ロボット工学の世界大会、コンピュータビジョンと AI システムの仕事、edge AI カメラ、そして SPRESENSE 上の embedded systems を通っています — NuttX と WireGuard は、その最新の到達点であって、 出発点ではありません。
- ここではっきりさせておきたいことがあります: 私は NuttX の kernel developer として 始めて、VPN を追加しようと思い立ったわけではありません。私は application 側・ deployment 側から NuttX に繰り返し出会い続けていて、ある時、その立場からは 解決できない課題に行き当たりました。それが今日の話です。

3 — 目次 (1:37 → 2:11)

画面: 7つの部。見出しは短い名詞句、その下に中身のキーワードを1行、右に所要時間 — Overview & background(~5分)/ The problem and the idea(~4分)/ The plan(~2分)/ Version 1: the app(~5分)/ Version 2: the kernel driver(~8分)/ Live demo(~3分)/ Summary & status(~4分)。

話す:

- 今日の話の地図です。
- 7つの部があります。まず概要と背景: NuttX とは何か、そして この取り組みの始まり。次に、解きたかった課題と、そのアイデア。3つ目が計画。 4つ目は第1版、アプリとしての実装: 最初のトンネル、隠れていた4つのバグ、 そして最初のデモ。5つ目は第2版、カーネルドライバと、その試験。6つ目は スタックちゃんでのライブデモ。最後にまとめと、作業が今どこにあるかです。
- 全部で約30分、その後質疑応答です。
- この後、部が変わるたびにこの目次に戻ってきて、今どこにいるかを示します。

4 — 目次 ①Overview & background (2:11 → 2:21)

画面: 目次と同じリスト。①Overview & background だけハイライト、他は薄い。

話す(一言): では、第1部の概要と背景から。

間: ハイライトされた行を指して一拍。

5 — プロジェクト概要 (2:21 → 3:01)

画面: `wg0` netdev の図 — アプリケーション/NSH、NuttX ネットワークスタック、 `eth0`/`wlan0` の隣に `wg0`、その下に UDP ソケット、インターネット、ピア。

話す:

- 全体像を1枚の絵にするとこうなります。
- Apache NuttX は独自の TCP/IP スタックを持つ POSIX 準拠の RTOS です。VPN は ありませんでした。このプロジェクトは `wg0` を仮想ネットワークインターフェース として追加します — アプリケーションからは普通のインターフェースに見え、その下では暗号化された伝送路が UDP ソケットです。反対側のピアは本物の WireGuard エンドポイント — Linux、Windows、あるいは別のゲートウェイです。

6 — 経緯: 2月の思いつきから10月の本番まで (3:01 → 3:53)

画面: 2月〜10月の月の横軸。上に出来事 — 2月 中旬 GSoC のテーマ一覧を見て調べ始める / 3月 上旬 計画を提案 / 3月20日 Alan さんからこのカンファレンスを聞き、講演を申し込む / 4月 上旬 GSoC に応募 / 5月 上旬 講演は採択、GSoC は不採択 — それでも続ける / 10月 グラスゴー。下に開発 — 6〜7月 少しずつ / 8月の休み(お盆)に開発の大半 / 9月〜 FLAT のアプリからカーネルドライバへ。

話す:

- このプロジェクトがどう進んだかを、月ごとに見せます。
- 2月の中旬に GSoC のテーマ一覧を見て、調べ始めました。3月の初めに計画を Issue に まとめて提案しました。3月20日に Alan さんからこのカンファレンスのことを聞いて、 講演を申し込みました。4月の初めに GSoC に応募しました。
- 5月の初めに2つの返事が来ました。講演は採択、GSoC は不採択。それでも作ることに しました。
- 6月からは少しずつ進め、まとまった開発は8月の休みにやりました。9月には FLAT の アプリからカーネルドライバに作り直しました — それがこの話の後半です。

7 — Background: NuttX とは何か (3:53 → 4:41)

画面: ソフトウェアスタック図(Applications / POSIX・ANSI・BSD socket API / Networking・VFS・filesystems・device drivers・NSH shell / NuttX kernel / MCU・MPU)。下にタグ: RTOS, POSIX-oriented, BSD sockets, Filesystem/VFS, NSH shell, Native networking, Device drivers, Apache Software Foundation TLP。

話す:

- 本題に入る前に、NuttX そのものを説明します。
- NuttX は、標準準拠を第一に置いたリアルタイム OS です。POSIX・ANSI API に準拠し、ネットワークは BSD socket で話し、本物のファイルシステムと VFS 層を持ち、NSH というシェルがあり、ネイティブなネットワーキングとデバイスドライバのモデルを 持ちます — これら全部を、8 ビットから 64 ビットのマイクロコントローラで動く サイズまで縮めています。NuttX は Apache Software Foundation の top-level project でもあり、kernel も applications も documentation も、contributor community に よって公開の場で開発されています。
- ここで言いたいことは一つに絞ります: NuttX は小さな RTOS でありながら、Unix 的な プログラミングモデルを持っている。この組み合わせは珍しく、この後出てくる socket、netdev、POSIX の形をした porting が可能なのは、すべてこれのおかげです。

8 — Positioning: NuttX の立ち位置と実際の採用例 (4:41 → 5:47)

画面: 左に RTOS 比較表(FreeRTOS / Zephyr / NuttX、design center と typical use)、GitHub スター数(FreeRTOS/FreeRTOS 7.8k・zephyr 16.5k・apache/nuttx 4.1k、「スターは可視性であって導入量ではない」の注記)。右に数字カード(15+アーキテクチャ、300+ボード、1500+コンフィグ、Sony / PX4 drones / Xiaomi OpenVela 1000+SKU / 日本の2024年月面ミッション)。

話す:

- 30秒だけ、NuttX がどこに位置するか、そしておもちゃのプロジェクトではないか、という話をします。
- 3つとも有力な RTOS で、重心が違います。FreeRTOS は意図的に小さいリアルタイム カーネルで、MCU ファームウェアや制御、IoT 向け。Zephyr は接続デバイス向けの 統合プラットフォームで、ウェアラブルや産業用・接続 IoT 向け。NuttX は Unix 的な 方 — ネットワーキング、ファイルシステム、ドライバがあり、ドローンやセンシングで 使われます。優劣表ではなく、形の違いです。GitHub のスター数で言えば、NuttX は 3つの中で一番小さい、とはっきり言います。スター数は開発者からの可視性であって、実際の導入量ではありません。
- 導入面は実在します: 15 以上のアーキテクチャ、300 以上のボード、1500 以上の コンフィグテンプレートが upstream にあります。実運用では Sony のデバイス、PX4 の フライトコントローラ、Xiaomi OpenVela の 1000 以上の SKU、そして日本の 2024 年の 月面ミッションでも飛んでいます。大きなエコシステムの隣では小さいですが、おもちゃではありません。

9 — History: Sony と NuttX の歴史 (5:47 → 6:49)

画面: 縦タイムライン — 2015 Sony audio products / 2018–19 SPRESENSE・CXD56xx (OSS+ELC Europe 2019 で発表) / 2020 upstream collaboration(NuttX Online Workshop、「A Journey from Fork to Mainline」) / 2020s sensing & edge devices。

話す:

- これは Sony の NuttX との歴史であって、まだ私の話ではありません — そして edge AI より長いです。
- Sony は 2015 年ごろから、NuttX を使った音声製品を出荷しています。数年後に登場した SPRESENSE とその CXD56xx チップは、音声、GNSS、カメラ、マルチコアのサポートを NuttX に持ち込み、2019 年に Sony の Masayuki Ishikawa 氏が Open Source Summit + Embedded Linux Conference Europe でその成果を発表しています。そこから、Alin Jerpelea 氏を含む Sony のエンジニアたちが、upstream への大きな貢献者に なっていきます — 後に NuttX の project management committee の議長も務める 人物です。彼の 2020 年の講演は文字通り「A Journey from Fork to Mainline (fork から mainline への旅)」という題でした。
- 一人のエンジニアがいたから Sony が NuttX を採用した、という因果関係は主張しません — それを裏付ける一次情報がないからです。公開されている講演や文書から言えることは もっとシンプルです: Sony の NuttX 利用は edge AI より何年も前からあり、カメラや GNSS、センシングの仕事へと時間をかけて広がっていった。これが、私自身の経験が 立っている土台です。

10 — Experience: SPRESENSE と AITRIOS での私の経験 (6:49 → 7:46)

画面: SPRESENSE と AITRIOS / edge AI カメラの実写真。下に "application development → system integration → deployment & operation" の フロー。

話す:

- 今のが Sony の歴史です。これは私自身の話です — 同じハードウェアファミリーですが、立ち位置が違います。
- SPRESENSE では、NuttX SDK を使って、センシングとカメラ、GNSS・LTE まわりの プロトタイピングと embedded 開発をしています。AITRIOS の edge AI カメラでは、ESP32 と NuttX で構築されたカメラを、ベンチの上ではなく実際に現場に設置して 動かす、という仕事です。
- 共通しているのは、私が NuttX に近づく方向です: application 開発、そして system integration、そして deployment・運用。これまで、そのために NuttX の ドライバを自分で書く必要は一度もありませんでした — 今回のプロジェクトまでは。そして私をそこへ連れてきた課題は、「NuttX をどう動かすか」ではありませんでした。「デプロイされた後の NuttX デバイスに、どうやって届くか」でした。

11 — 目次 ②The problem and the idea (7:46 → 7:56)

画面: 目次と同じリスト。②The problem and the idea だけハイライト、他は薄い。

話す(一言): 背景は以上です。第2部: 解きたかった課題と、そのアイデア。

間: ハイライトされた行を指して一拍。

12 — Problem: 遠隔アクセスの隙間 (7:56 → 8:48)

画面: Engineer → Internet → Router/network → Wi-Fi → NuttX device の図。WPA2/WPA3 が守る範囲(Wi-Fi の一区間のみ)と、必要な認証済み end-to-end パスの 対比。下に3枚のカード: open it to the Internet / make your own protocol / use a vendor's cloud。

話す:

- ここからは課題そのものを、率直に話します。

- Wi-Fi のセキュリティは本物ですが、守っているのは一区間だけです: デバイスと アクセスポイントの間の無線リンク。それより手前 — 技術者、インターネット、ルーター — は、WPA2 や WPA3 が約束できる範囲の外です。私が必要としていたのは、経路全体 — 技術者からデバイスまで — の認証と暗号化で、最後の数メートルだけ ではありません。
- それがないと、実務的な選択肢はどれも今より悪くなります: デバイスにグローバル IP を公開して祈る、独自のセキュアプロトコルを発明して保守し続ける、あるいは 接続をベンダーのクラウドに委ねる。同じ隙間は、edge AI や産業用 IoT、遠隔 インフラ全般にも現れます — このプロジェクト固有の話ではありません。

13 — 他の手段との比較 (8:48 → 10:05)

画面: 7つの手段(インターネットにポートを開ける / クラウド API への HTTPS / MQTT ブローカー / リバーストンネル・リレー / ルーターに VPN / キャリアの閉域網 / 機器に VPN)を、誰から接続するか・任意のサービスに届くか・間にサーバーが要るか・ どこまで守られるか・NuttX に今あるもの、で比べた表。最後の行(機器に VPN)が今回の話。

話す:

- WireGuard を選ぶ前に、機器に届くための一般的な方法を比べました。
- ポートを開けるのは簡単ですが、インターネット全体から届いてしまいます。HTTPS と MQTT は多くの IoT 製品が使っている方法です。機器からサーバーへ接続しに行くので NAT は問題にならず、NuttX にも両方のクライアントがすでにあります。ただし届くのは 自分で設計したメッセージだけで、機器にログインしたり Web ページを開いたりは できません。暗号化もブローカーやクラウドで終わります。
- リバーストンネルは、リレーサーバー経由でポートを1つ通します。ルーターの VPN は ルーターが自分のものなら良い方法ですが、LAN の最後の一区間は守られず、顧客の 現場ではルーターは普通こちらのものではありません。キャリアの閉域網はセルラー 回線だけで、契約が要ります。
- 機器の上の VPN だけが全部を満たします: 任意の IP サービス、間にサーバーなし、機器までの暗号化。データのやり取りには今でも MQTT や HTTPS が正しい道具で、これは保守のためのアクセスです。

Tailscale や ZeroTier について聞かれたら: これらも機器上の VPN だが、今のところ マイコンより大きな OS が要ることが多い(スライドの脚注)。

14 — Solution: なぜ WireGuard なのか (10:05 → 10:58)

画面: Engineer/Linux/Windows ↔ WireGuard トンネル ↔ NuttX device の図。下に3点: modern IP-layer VPN / UDP transport, public-key peers / already everywhere。

話す:

- その課題に対して、なぜ WireGuard なのか、です。
- WireGuard はモダンな IP 層の VPN です: 設計上小さくシンプルで、IPsec や OpenVPN に比べてコードサイズがごく一部です — これはマイクロコントローラの 上では大きな意味を持ちます。素の UDP の上で動き、ピアは証明書や交渉された セッションではなく公開鍵で識別されます。正しい鍵を持たないパケットには 一切返事をしません — 外から見れば、ただのレンガの壁です *(9¾ 番線へのオマージュ。知らなくても普通の比喩として通じる)*。そして重要なのは、特殊なものでは ないということです: 何年も前から Linux カーネルに mainline で入っていて、公式の Windows クライアントがあり、トンネルの反対側は既存のどんな WireGuard ピアでも構いません。
- つまり選んだのは「製品」ではなく、「独自プロトコルを発明する、あるいは ベンダーのクラウドに頼る代わりに、標準プロトコルを採用する」という選択です。これが、この特定の課題に WireGuard が合う理由です。

15 — Architecture: NuttX ネットワークスタック上の WireGuard (10:58 → 11:46)

画面: Application → TCP/UDP/IP → wg0 (NET_LL_TUN netdev) → WireGuard encrypt/decrypt → UDP socket → wlan0/usrsock・物理ネットワーク、の層状図。左右に TX/RX 両方向の矢印。

話す:

- 概要図より一段技術的になります。ここから実装の話が始まります。
- 送信方向では、アプリケーションから TCP または UDP・IP を通って降りてきた パケットが、他の netdev と同じように `wg0` に届き、WireGuard によって 暗号化され、ボード上では `wlan0` を通る、あるいは Wi-Fi そのものがソケット ベースの周辺機器である `usrsock` を通る UDP データグラムとして出ていきます。受信方向はこの逆です: UDP データグラムが届き、WireGuard が復号し、普通の インターフェースに届いたかのように `wg0` へ渡されます。
- ここから先はすべて、この真ん中の部分 — `wg0` と暗号化のステップ — を、上下の層に触れずに作る話です。

16 — 目次 ③The plan (11:46 → 11:56)

画面: 目次と同じリスト。③The plan だけハイライト、他は薄い。

話す(一言): 第3部: 計画。既存の WireGuard をどう再利用するか。

間: ハイライトされた行を指して一拍。

17 — Porting Strategy: wireguard-lwip を lwIP なしで再利用する (11:56 → 12:57)

画面: 3段カード — Protocol & crypto(KEEP、緑)/ Platform hooks(ADAPT、金、 `wireguard-platform.h` の4関数)/ lwIP network glue(REPLACE、赤)。

話す:

- ゼロから暗号スタックを書くのは間違いです。なので `smartalock/wireguard-lwip` という既存実装から始めました、その理由です。
- `wireguard-lwip` はきれいに3層に分かれています。プロトコルと暗号 — ハンドシェイク、トランスポート、BLAKE2s、ChaCha20-Poly1305、Curve25519 — は OS 依存のない移植可能な C です。暗号は無変更のまま使いました。プロトコル本体は、後のレビューで小さな修正が2つ必要になり、PATCHES.md に書いてあります。OS 依存の 振る舞いはすべて、すでに1つのヘッダー、`wireguard-platform.h` の裏に、4つの関数として隔離されていました。置き換えが必要だったのは lwIP の ネットワークグルーの部分だけです — lwIP の netif を登録し、lwIP のバッファを 管理する部分です。NuttX には lwIP がそもそも存在しないからです。
- 何も書き直さずに済んだのは運ではなく、きれいなプラットフォーム境界が もたらすものです。その境界があったからこそ、最初の移植作業は書き直し ではなく差し替えで済み、作業量そのものが桁違いに小さくなりました。

18 — Design: lwIP を NuttX netdev へ写像する (12:57 → 13:19)

画面: lwIP → NuttX の対応表(`netif` → `net_driver_s`、`pbuf` → `iob`、 `netif_add()` → `netdev_register()`、コールバック → `devif_poll()`)。

話す:

- 移植の中で最も重要な判断です。
- lwIP を NuttX に引きずり込むのではなく、抽象化を NuttX 自身のものに写像 しました。`netif` は `net_driver_s` に、`pbuf` は `iob` に、`netif_add` は `netdev_register` に、送信コールバックは NuttX の `devif_poll` になります。 `wg0` は TUN 型の netdev として登録され、その下の「配線」は UDP ソケットです。

19 — 目次 ④Version 1: the app (13:19 → 13:29)

画面: 目次と同じリスト。④Version 1: the app だけハイライト、他は薄い。

話す(一言): 第4部: 第1版、アプリとしての実装。動いた、そしてバグが出てきました。

間: ハイライトされた行を指して一拍。

20 — Result: 動くトンネルは始まりにすぎない (13:29 → 14:14)

画面: 表(Protocol & crypto: 暗号はバイト単位で同一、プロトコル本体は小さな修正2つ(PATCHES.md) / OS hooks: 4関数 / Network glue: 実際の作業)。`wg genkey / set / setconf` が実行時に可能、という claim。

話す:

- 最初のバージョンの結果です。
- プロトコルと暗号処理、3000行以上を持ってきました。暗号はそのままです。プロトコル本体はレビューの後で小さな修正が2つ必要でした — そのうち1つは wireguard-lwip 自身のバグで、レート制限が一度も効いていませんでした。4つの OS フックは小さなものです。本当の作業はネットワークの つなぎ込みでした。そして動きました: 鍵の生成、ピアの設定、設定ファイルの 読み込み、すべて実行時に、再ビルドなしで。
- ここまでは成功譚に見えます — でも、お姫様は別の城にいました *(マリオの「Your princess is in another castle」)*。このスライドの本当の役割は、そこです —「動いた」ように見えるところまでが実は一番安く手に入る到達点で、この後の話はほとんど、その「動いた」が何を隠していたか、です。

21 — Where it was hard: 苦労したところ (14:14 → 14:39)

画面: 見出し「The tunnel worked fast. The bugs took the time.」。フロー図 — SIMPLE TEST (handshake/ping、✓PASS)→ ASSUMPTION(「そこは正しいはず」)→ DEEPER TEST(TCP/real HW/kernel/ security、✗FAIL)。下に「Six bugs took most of my time.」。

話す:

- ここからは、苦労したところです。
- 最初のトンネルはすぐに動きました。時間の大半を使ったのは6つのバグです。apps 版で4つ、カーネルに移してからさらに2つ。多くは、ハンドシェイクや ping のような通る試験の陰に隠れていたので、最初は違う場所を探していました。まず第1版の4つです。

22 — 落とし穴 1/6: detach された pthread(最初の症状: recvfrom が戻らない) (14:39 → 15:45)

画面: 左の先頭に1行 — 最初の症状: `recvfrom()` が戻らず、`SO_RCVTIMEO` のせいだと思った。本当の原因はここ。

話す:

- 1つ目: 成り立たなかったライフサイクルの前提。
- 最初の症状は単純でした: `recvfrom` が戻らず、タイマーが動かず、ハンドシェイクも送られない。受信タイムアウトのオプションのせいだと思って `poll` に替えましたが、それでも何も送られませんでした。本当の原因はここでした。 受信スレッドは detach 付きで作られていました。起動したコマンドが終了すると、スレッドも止まりました — `ps` にすら出てきません。作成直後に短い sleep を 挟むと `printf` が実行されたので、作成自体はされていました。私の勘違いは detach を「コマンドより長生きする」と読んだことです — detach が制御するのは スレッドのリソースがいつ回収されるかだけで、起動元が終了しても生き残るか どうかではありません。だからコマンドが終了すると、スレッドも道連れに なりました。カーネル版でスレッドをドライバ側が所有しているのは、まさに これが理由です。なお、NuttX の UDP のコードはそのタイムアウトを使っています。 古い経路はその後試し直していません。

23 — 落とし穴 2/6: ping は通るのに TCP が死ぬ (15:45 → 16:37) *(お気に入り)*

話す:

- このプロジェクトで一番好きなバグです。
- 同じトンネルの上で、ping は完璧に通り、TCP の 3-way ハンドシェイクすら 完了する — それなのに接続は死にました。送信経路を計装すると `EBADF`、不正なファイルディスクリプタが出ていました。成功していた送信はすべて 受信タスク自身のコンテキストで行われたものでした — ハンドシェイクの 応答、キープアライブ、ping の応答、SYN-ACK。他のタスクからの送信はすべて 失敗していました。
- 原因: NuttX ではファイルディスクリプタはタスクグループ単位にスコープされ ていて、システム全体でもグローバルでもありません。修正は fd を経由する こと自体をやめ、NuttX の内部ソケット API を使うことでした — これは `struct socket` に対して動作する、ただのメモリなので、どのタスクからでも 使えます。カーネル版がソケットを `struct socket` として保持しているのは、まさにこの理由です。

24 — 落とし穴 3/6:「実害なし」が実は10倍 (16:37 → 17:17)

話す:

- 3つ目: 測定しなかった前提。
- fd を使わない修正は `poll` も壊しました。`poll` は fd ベースだからです。 応急処置として 50ミリ秒ごとにソケットをポーリングしました。コストは アイドル時に CPU を起こすことだけだと思っていました。間違いでした。 届いたパケットはすべて最大 50ミリ秒待たされ、それだけでスループットの 上限になっていました — 本来より約10倍遅い。「実害なし」と測ったものが 実は10倍でした。本当の修正は正しいブロッキング待ち — `psock_poll` への コールバックとセマフォです。

25 — 落とし穴 4/6: sim は通るのに実機は違う (17:17 → 18:11)

話す:

- 4つ目、そして後で戻ってきます: 誰も測っていなかったスタック。
- 実機でしか出なかったことが2つあります。1つ目: シェルのリダイレクトで 設定をファイルに保存すると、実機の flash ファイルシステムでのみボードが 固まりました — sim では一度も再現しません。ファイルを自分で書く `wg saveconf` を足して解決しました。
- 2つ目は受信タスクのスタックです。IP スタックへの呼び出しは自分自身の スタックの上で行われるので、TCP の呼び出し深さ全体がそこに乗り、即座に応答するパケットについては返信の暗号化もそこに乗ります。既定値の 3072バイトは、誰も測ったことのない数字でした。スタックカラーレーション と `ps` で実機で測ると94%使っていて、残りは176バイトだけでした。この話は後で戻ってきます。

26 — 第1版のデモ: 最初の動画 (18:11 → 18:37)

画面: 左に動画の1コマ(公式 Windows WireGuard クライアントと、ESP32-S3 上の NuttX への telnet セッションが並んだ画面)。右に手順 — telnet 10.10.0.2 / uname -a、ifconfig / webserver & / ブラウザで http://10.10.0.2/ — と条件(ESP32-S3 DevKit、apps 版 (FLAT)、USB は抜いてある、相手は 公式 Windows クライアント)、小さく youtu.be/1kyX2av5WG4。

話す:

- 第1版の動画の1コマです: トンネル越しに telnet でログインし、コマンドを いくつか打ち、ボード上の小さな Web ページを開きました。ボードは電源アダプタだけで 動いていて、相手は公式の Windows 版 WireGuard です。第1版でもデモには十分でした。 次に、これを本物の NuttX のドライバにしました。

再生しない: 静止画で十分。動画はライブデモの予備として取っておく。**これは apps 版。** この後のカーネル版の結果と混ぜて話さない。

27 — 目次 ⑤Version 2: the kernel driver (18:37 → 18:47)

画面: 目次と同じリスト。⑤Version 2: the kernel driver だけハイライト、他は薄い。

話す(一言): 第1版は以上です。第5部、難しい後半: 本物のカーネルドライバと、その確かめ方。

間: ハイライトされた行を指して一拍。

28 — 第5部: カーネルへ (18:47 → 19:23)

話す:

- ここからが最初のバージョン以降の追加作業で、upstream に入れるには、NuttX ではこの部分が必要です。
- バージョン1は `apps` に住み、ソケットの内部実装に踏み込んでいます。FLAT ビルドなら問題ありませんが、正式な in-tree ドライバには決してなれず、ユーザー空間とカーネルが分離される PROTECTED / KERNEL ビルドにも合いません。そこでデバイスを `drivers/net/wireguard` としてカーネルに移し、小さな ユーザー空間の `wg` コマンドから ioctl 経由で操作する形にしました。

29 — カーネル側の設計 (19:23 → 20:20)

話す:

- 実際のドライバの形です。
- `wg0` は今やカーネル内の lower-half netdev で、UDP ソケットと受信カーネル スレッドを自分で所有します — なので前の章の「ソケットの所有権」と「起動元の 寿命」の問題は設計上そもそも起きません。設定は平坦でポインタを含まない ioctl ABI です。ポインタを含まないので、カーネルは呼び出し元が渡した1つの構造体を 呼び出しの間に読むだけで、ユーザーメモリへのポインタをたどる必要がありません。秘密鍵は write-only — get の ioctl は決して それを返しません。ただしユーザー側の設定ファイルには残っています。カーネル側の暗号処理は NuttX 自身のものです。プロトコル本体は今も wireguard-lwip 由来の BSD-3-Clause のコードなので、ドライバには ALLOW_BSD_COMPONENTS が必要で、そのことを LICENSE ファイルに書いて あります。ユーザーコマンドには、オフライン鍵生成用の小さな MIT ライセンスの X25519 もあります。

30 — 落とし穴 5・6/6: もう2つのバグ・同じ形 (20:20 → 21:22) *(29と一体)*

話す:

- そしてカーネルの深さでも同じパターンが2回起きました。
- 1つ目は NuttX 自身の ChaCha20-Poly1305 のバグです: カウンタがナンスの 誤ったバイト位置に入っていました。カウンタ0の場合は両方のレイアウトが 一致するので、最初のデータパケットだけは通り、2つ目以降がすべて復号に 失敗します。in-tree の 呼び出し元が一度もなかったので、これまで一度も踏まれていませんでした。修正は2行で、ドライバとは別の `crypto:` の変更として分けてあります — この プロジェクトへの移植作業が、このプロジェクト自身の潜在バグを見つけました。
- 2つ目は、カーネルビルドでしか起きないクラッシュです。鍵の設定がピアを 6キロバイトの配列としてスタック上にスナップショットしていて、カーネルの スタックは3キロバイトしかありません — オーバーフロー、ヒープ破壊、 パニック。タスクスタックが大きい sim では一度も現れませんでした。まさに前の章のスタックの教訓が、もう一段深いところで再現したものです。ヒープに移しました。

31 — 検証: 成り立つべき7つの場所 (21:22 → 23:35) *(核心・削らない)*

画面: sim / rv-virt knetnsh64 (BUILD_KERNEL, QEMU) / rv-virt pnsh64 (BUILD_PROTECTED, QEMU) / rv-virt knetnsh64_smp (4 CPU, QEMU) / SPRESENSE 実測(FLAT) / ESP32-S3 + SPRESENSE 実 Wi-Fi(FLAT。実機の2行は最終 rebase 前の版)/ 両ボードでの Version 1 (app), v0.1.1、の表。

話す:

- どの版をどこで確かめたか、正確に話します。
- カーネルドライバは sim 構成でエラーなくビルドできます — この構成は ドライバのシリーズに入っています。そして QEMU 上の カーネルビルド、rv-virt の knetnsh64 で、Linux カーネルの WireGuard と完全なトンネルを 張ります。`wg` コマンドは別のプログラムで、システムコールでドライバに届きます。
- 次に PROTECTED、これも QEMU です。ioctl ABI の中にはポインタがないので、カーネルは 呼び出し元が渡した1つの構造体だけを触ります。メモリ分割を有効にして、すべての ioctl が 境界を越え、トンネルは Linux まで届きます。繰り返し実行すると、8回中2回、例外が 起きました。原因はスタックあふれでした: ここではシステムコールが呼び出し側のスタックの上で 動き、`wg` のスタックはそれには小さすぎました。既定値を大きくすると、QEMU で20回中20回成功しました。
- そしてロックです: QEMU 上の4 CPU で、パケットを大量に受けながら、別の CPU が設定を 変え、down と up をします。例外はなく、トンネルは戻りました。ただしこれはスクリプト化 した1回の実行で、レース条件をくまなく探したものではありません。

- SPRESENSE の行は、査読者に最初に見せたい行です。シミュレータのメモリの数字は ボードには当てはまらないので、ボード自身を測りました: 2分間の通信、その後実 Wi-Fi で 40回の down/up。ヒープ、バッファ、すべてのスタック: 前後で完全に同じ。受信スレッドが 使ったのは、既定の6キロバイトに対して1472バイトでした。この既定値はかつては推測でした。今は実測値です。
- カーネルドライバは両ボードでも、実 Wi-Fi 越しに公式 Windows クライアントと動いて います: ESP32-S3 はネイティブ Wi-Fi、SPRESENSE は usrsock。これは通常の FLAT ビルドで、最終 rebase の前の版です。KERNEL と PROTECTED は QEMU でだけ動かしていて、KERNEL と SMP のトンネルは rebase の後にもう一度確認しました。最後の行は第1版、アプリを両ボードで 動かしたものです。

32 — 検証: 動くトンネルは証拠にならない (23:35 → 25:07) *(核心・削らない)*

画面: 3行の表 — 毎回同じ秘密鍵 / トンネルより長生きするセッション鍵 / 数時間経たないと現れないリーク — それぞれ「pingが通れば気づくか」に「気づかない」。測った場所のラベル付き(SPRESENSE / sim / sim)。

話す:

- 前のスライドはどこで動くかでした。このスライドは、動くトンネルが教えてくれないことです。
- まず鍵です。NuttX は `/dev/urandom` に、ビルド時の設定から seed を取る xorshift128 backend を選べます。そうしたボードは起動のたびに同じ秘密鍵を配り、それでもハンドシェイクも ping もデモも動きます。そこで SPRESENSE を3回リセットし(電源の入れ直しではなくリセット)、3つの異なる鍵を得ました。簡単な確認であって、乱数の質の証明ではありません。
- 2つ目はシミュレータです: トンネルが up の間と down の後に、デバイスのメモリを 読みました。セッション鍵は down の後すべてゼロでした。静的鍵は、もう一度 up できるように 意図的に残しています。複製はちょうど1つで、そのことはドライバの文書に書いてあります。
- 3つ目、長時間の試験で、これもシミュレータです: 鍵の寿命を超えて強制した151回の 再鍵化、10回のエンドポイント変更、100回の down/up。バッファはすべて戻り、生きている アロケーション数は起動時とちょうど同じに戻りました。受信スレッドのスタック高水位は シミュレータで2.5キロバイト — ボードでは1.5キロバイト — で、既定は6キロバイトです。主張していないのは、1週間動いたこと、そしてシミュレータのメモリがボードと同じように 振る舞うことです。

33 — 設計判断: タイムスタンプ問題には無料の答えがない (25:07 → 27:03) *(核心・削らない)*

画面: リプレイ防御が課す義務、4つの選択肢の表(リアルタイム+起動内 高水位 / 毎回永続化 / 耐久性のある範囲予約 / 追いつくのを待つ)、実測の ペア(時計未設定 → 75秒見ていてハンドシェイクなし、時計設定 → 4.1秒)。

話す:

- ここは試験ではなく、判断が必要だった唯一の場所です。そして、その判断を測りました。
- WireGuard のリプレイ防御は、ハンドシェイクを始める側に規則を課します: すべての ハンドシェイクは、相手がこれまでに見たものより新しいタイムスタンプを持たなければ ならない — 再起動の後でもです。デバイスは普通、始める側で、しかもバッテリー付きの 時計を持っていない可能性が最も高い側です。
- 何もしないのは駄目です。稼働時間ベースのタイムスタンプだと、デバイスは前回動いて いた時間と同じだけ切断されます: 1か月動いていたデバイスは、1か月再接続できません。毎回のタイムスタンプを保存すると flash が摩耗します。flash に範囲を予約するのが長期的 には正しい答えですが、それには、これらのファイルシステムで確かめられなか

った書き込みの 保証が必要です。それを出荷すれば、解決したように見えて解決していないことになります。ここでは、楽な選択と正しい選択が一致しませんでした *(ダンブルドアの「正しいことと楽なこと」)*。

- そこで、リアルタイムクロックと、この起動の中で使った最大値を選びました — Linux と wireguard-go とまったく同じです。私が足したのは: 時計が一度も設定されていなければ、ドライバが一度だけ警告を出し、何が起きるかを書くことです。
- そして実機で測りました。素朴な試験は通ってしまいます: ボードをリセットして ping すると、トンネルは戻ります — ping が相手側にハンドシェイクを始めさせたからです。そこで ボードには何も送りませんでした。同じボード、変えたのは1つだけ。起動時のままの時計: 警告が出て、75秒見ていてもハンドシェイクはありません。時計を設定すると: 4.1秒で ハンドシェイク。時計がなければ、デバイスは自分から再接続できないかもしれず、ping の 試験ではそれが見えません。だからドライバは警告を出します。

34 — 目次 ⑥Live demo (27:03 → 27:13)

画面: 目次と同じリスト。⑥Live demo だけハイライト、他は薄い。

話す(一言): ここまでは技術の話でした。第6部: 実際に動くところを見てもらいます。

間: ハイライトされた行を指して一拍。

35 — デモ構成: スマホのネットワーク上の1本のトンネル (27:13 → 28:11)

画面: 構成図。会場 Wi-Fi(機器どうしの通信を遮断)→ スマホのテザリング (独自の小さなローカルネットワーク)→ ラップトップ(Docker 内の WireGuard ピア、wg0 10.10.0.1、音声ファイルを :8000 で配信)とスタックちゃん (ESP32-S3 + NuttX、テザリング上の wlan0 越しに wg0 10.10.0.2。apps 版 v0.1.1、FLAT ビルド)を 1本の WireGuard トンネルで結ぶ。SPRESENSE は点線の2台目のピア(wg0 10.11.0.2)。

話す:

- デモの構成です。
- ラップトップとスタックちゃんは、どちらも私のスマホのテザリングにつないで います。会場の Wi-Fi は使えますが、機器どうしの通信を遮断しているので、スマホに小さなローカルネットワークを作ってもらいます。スマホ自身の インターネットは会場 Wi-Fi でもモバイル回線でもよく、デモには必要ありません。
- ラップトップとスタックちゃんの間に WireGuard のトンネルが1本あります。ラップトップ側はコンテナ内の普通の Linux WireGuard で 10.10.0.1、スタックちゃんは 10.10.0.2 です。telnet も Web ページも、音声さえも、デモのすべてがこのトンネルを通ります。音声は事前にラップトップで作ったもので、クラウドの API はどこにも使っていません。
- 見てほしいのは2つです。ログインした後のアドレスと、パケットです。トンネルの外では、暗号化された UDP しか見えません。

36 — デモ (28:11 → 29:41)

画面: 7つの手順 — wg show / telnet 10.10.0.2 / ifconfig / stackchan face happy / stackchan say hello.wav / tcpdump の外と内 / もう一度 wg show。右側: 外 = 暗号化された UDP 51820 だけ、内 = telnet の文字列、「apps 版 (v0.1.1)、FLAT ビルド」の表示。

話す:

- このデモの要点は、特別なことが何も起きていない、ということです。スタックちゃんは、暗号化トンネルの向こう側にある普通のコンピュータです。このボードは apps 版で動いています — 動画と同じアプリです。カーネルドライバも このチップで動きましたが(スライド31)、ロボットのデモは apps 版の上に作っています。
- まずラップトップで wg show。ハンドシェイクが済んでいて、スマホの ネットワーク上のスタックちゃんのアドレスが見えます。次に 10.10.0.2 へ telnet します — これはトンネルの中のアドレスです。ifconfig を打つと両方が 見えます。スマホのネットワーク側の wlan0 と、トンネルの中の wg0。つまり、本当にトンネル越しにログインしています。
- これで操作できます。笑顔にして、hello と言わせてみます。声もラップトップから 同じトンネルを通って届いています。
- 最後にパケットです。左はスマホのネットワーク: 暗号化された UDP だけ。右はトンネルの中: 同じ telnet のやり取りが平文で見えます。もう一度 wg show すると、転送バイト数が増えています。Mischief managed(いたずら完了) *(忍びの地図の合言葉。デモの締めの一言として軽く)*。

ライブ: `scripts/stackchan/demo.sh` を使う。左の端末で `login`、右の2つの端末で `outside` と `inside`、前後に `show`。画面は左に端末、右にスタックちゃん(カメラか机の上)。

本番前: スマホのテザリングを ON(2.4 GHz / 互換性優先)、ラップトップを 接続、Docker Desktop を起動。スタックちゃんを `nsh_wifi.py` でつなぐ (パスフレーズはボードに保存しない)、`wg_setup.py` を1回実行。 `stackchan say` を1回試す。テザリングを入れ直すとアドレス帯が変わることが あるので、そのときは `nsh_wifi.py` と `wg_setup.py` をやり直す。 `say` の途中でボードが止まることがまれにある(約20回に1回)。止まったら 電源を入れ直して同じ手順。

画面に出さない: `wg showconf`(秘密鍵を表示する)、 `%USERPROFILE%\stackchan-wg` の鍵ファイル、テザリングのパスフレーズ。

フォールバック: 録画。ライブが止まったらすぐ切り替える。

37 — 目次 ⑦Summary & status (29:41 → 29:51)

画面: 目次と同じリスト。⑦Summary & status だけハイライト、他は薄い。

話す(一言): 最後の部: 作業が今どこにあるかと、持ち帰ってほしいこと。

間: ハイライトされた行を指して一拍。

38 — 運用性 (29:51 → 30:25) *(削る候補3)*

話す:

- 「デモで動く」から「運用できる」への移行です。
- 鍵はもうビルドに焼き込まれていません — 実行時に生成・設定でき、ファームウェアイメージには一切残りません。長時間稼働のために、4つの独立した信号を毎分サンプリングします — ルーター側が生きているか、ボードがネットワーク上にあるか、WireGuard が運んでいるか、アプリ ケーションのデータが流れているか — なので静かになったとき、推測ではなくどの層が壊れたかを言えます。

39 — 移植性 (30:25 → 30:53) *(削る候補2)*

話す:

- 最初からの設計判断を、検証した結果です。

- OS 依存部分を数個の関数の裏に隔離するのは賭けでした。これがその 検算です。apps の FLAT 実装で: x86、ARM Cortex-A7、ESP32 上の Xtensa、SPRESENSE 上の Cortex-M4F という4つのアーキテクチャの 間を、コード変更なしで移動できます。カーネルドライバはこの4つの 関数を使わないので、この確認は apps 版についてのものです。賭けは報われました。

40 — 還元する: 今の状態 (30:53 → 31:34) *(おまけ・最優先で削る)*

話す:

- この場の価値観に話を閉じます。
- 作業は小さな単位に分けてあります: それだけで役に立つ小さな暗号の修正、カーネルデバイスのドライバ、ユーザー空間のコマンド。どのコミットも patch の検査を 通り、1つずつ単独でビルドできます。まだ何も送っていません。各コミットには、まだ 私の署名と Assisted-by の行がありません — AI ツールに手伝ってもらったので、この行が要ります。最終的なシリーズは、ESP32-S3 の実機(普通の FLAT ビルド)でもう動いています。 作業が本当はどこにあるかを正直に言い、そのうえでレビューに変えてもらう。それが Community over code です。

「もうマージされたのか」と聞かれたら: まだです — これは提出候補 であって、マージ可能という宣言ではありません。詳細は Q&A 資料参照。

時間がなければ: このスライドはまるごと飛ばしてよい。本編の主張 (検証・パターン)は背負っていない。飛ばしても質疑で聞かれたら答えられる ように、上の2点だけ頭に入れておく。

41 — Summary: 作ったものと、覚えておいてほしい3つ (31:34 → 32:26)

話す:

- まとめます。
- 作ったのは wg0、NuttX のカーネルの中の WireGuard ネットワークデバイスです。 実機の ESP32-S3 と SPRESENSE では公式の Windows クライアントと、シミュレータと QEMU では Linux カーネルの WireGuard とつながり、小さな PR に分けて レビューを受けられる状態です。upstream にはまだ何も送っていません。
- そして覚えておいてほしいことが3つ。1つ目、ハンドシェイクの先まで試すこと: 長時間のデータ、実機、本物のカーネルビルド。2つ目、きれいな OS 抽象化 の境界は苦労するだけの価値があります — apps 版で、4アーキテクチャ、3バージョン、コード変更ゼロ。3つ目、あるプロジェクトに移植する作業は、そのプロジェクト 自体を改善する機会になります。1つのバグは NuttX 自身の暗号処理に あり、その修正はそれだけで成り立ちます。

42 — 謝辞: この発表を支えてくれた人たち (32:26 → 33:01)

画面: 4つの枠 — Alan / Apache NuttX コミュニティ / Apache Software Foundation と CoC の運営・ボランティアの皆さん / Sony の NuttX 開発者(これまでの、そして今の)。

話す:

- 最後に、お礼を言わせてください。
- まず Alan。このカンファレンスを教えてくれて、発表を出すよう背中を押して くれました。次に Apache NuttX コミュニティ。GSoC の提案を書いたとき、 話を聞いて励ましてくれました。採択されなかった後も続けられたのは、その おかげです。3つ目に、Apache Software Foundation と、Community Over Code を 運営してくださっている皆さん。そ

して最後に、私より前の Sony のエンジニアの 皆さん。NuttX を製品に入れ、プロジェクトに多くを還元してきました。この仕事は、その上に立っています。

43 — ありがとうございました (33:01 → 33:10)

画面: Code & records: `github.com/wwlapaki310/gsoc2026-nuttx-wireguard`。

話す:

- ありがとうございました。ご質問をお受けします — カーネルドライバ、バグ、ioctl ABI、ハードウェア、何についてでも。

その後: Q&A はスライドではなく coc-glasgow-qa.md を手元に。よくある質問(リベースの状況、#14、IOB 枯渇、checkpatch)は そこに答えがありますが、どのスライドにも載っていません。